Programación 1 — Unidad 9: Manejo de archivos

Actividad 2: Operaciones básicas con archivos

Índice

9. Manejo de archivos

– 9.4. Abrir, operar y cerrar

– 9.5. Los modos de apertura: r, w y a

– 9.6. Leer un archivo

– 9.7. Escribir en un archivo

– 9.8. La forma recomendada: with



9.4. Abrir, operar y cerrar

En la actividad anterior vimos **por qué** necesitamos archivos. Ahora vamos al **cómo**. Trabajar con un archivo tiene siempre tres momentos: **abrirlo**, **leer o escribir** en él, y **cerrarlo**. Se abre con la función `open()`, indicando el nombre del archivo y el **modo de apertura** (si vamos a leer, escribir o agregar).



Ciclo de vida de un archivo: abrir, operar (leer o escribir) y cerrar para liberar el recurso.

Cerrar no es un detalle menor. Mientras el archivo está abierto, el sistema operativo lo mantiene reservado y lo que escribiste puede estar todavía en un búfer, sin llegar al disco. `close()` vacía ese búfer y libera el recurso.

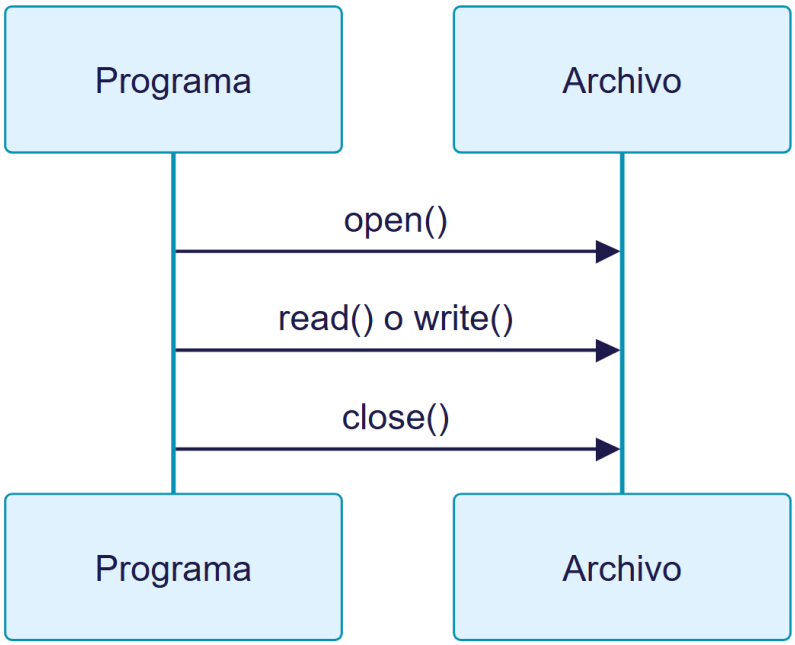


Diagrama 9.4: el programa le pide al archivo abrir, operar y cerrar.

```
archivo = open("notas.txt", "w")
archivo.write("Primera línea\n")
archivo.close()
```

Qué hace el código:

- `open("notas.txt", "w")` abre el archivo y devuelve un **objeto archivo**, que guardamos en la variable `archivo`.
- `archivo.write(...)` escribe texto adentro.
- `archivo.close()` cierra el archivo y asegura que lo escrito llegue al disco.
- Si te olvidás del `close()`, podés perder datos. Más adelante vamos a ver una forma de que Python lo haga por vos.

9.5. Los modos de apertura: r, w y a

El segundo argumento de `open()` es el **modo**. Determina qué podés hacer con el archivo y qué pasa con el contenido que ya tenía. Estos son los tres que vas a usar:

Modo	Significado	Comportamiento
`"r"`	read (lectura)	Lee; da error si el archivo no existe
`"w"`	write (escritura)	Crea o sobrescribe todo el contenido
`"a"`	append (agregar)	Agrega al final sin borrar lo anterior

Si no indicás el modo, Python usa `"r"` por defecto. Es decir, `open("datos.txt")` es lo mismo que `open("datos.txt", "r")`.

Importante

Mucho cuidado con el modo `"w"`: **borra todo el contenido previo** del archivo antes de escribir. Si querés conservar lo que ya había y solo sumar al final, usá el modo `"a"` (append). Confundirlos es una causa frecuente de pérdida de datos.

9.6. Leer un archivo

Una vez abierto en modo lectura, Python te da varias formas de traer el contenido. Cuál conviene depende de si necesitás todo junto o de a poco:

Método	Qué devuelve	Cuándo usarlo
`read()`	Todo el archivo en un solo string	Archivos chicos que querés procesar enteros
`readline()`	La línea siguiente, como string	Cuando querés avanzar de a una línea
`readlines()`	Una lista con todas las líneas	Cuando necesitás la lista completa en memoria
`for linea in archivo:`	Una línea por vuelta del bucle	**La forma recomendada**: sirve para cualquier tamaño

```
archivo = open("notas.txt", "r")

for linea in archivo:
    print(linea.strip())

archivo.close()
```

Qué hace el código:

- `for linea in archivo:` recorre el archivo **línea por línea**, sin cargarlo entero en memoria.
- Cada `linea` que llega incluye el salto de línea `\n` del final.
- `linea.strip()` quita ese salto sobrante para que el `print` no deje renglones en blanco de más.
- Cuando el bucle termina, ya se leyó todo el archivo: no hace falta contar las líneas ni saber cuántas eran.

Nota

Hasta ahora pensamos la eficiencia en términos de cuántas operaciones hace un programa. Con archivos aparece otro recurso a cuidar: la **memoria**. Si un archivo tiene un millón de líneas y usás `archivo.read()` para cargarlo entero en un string, el programa necesita tener todo ese texto en memoria al mismo tiempo. Recorrer el archivo línea por línea (`for linea in archivo:`) procesa una línea a la vez, sin necesitar el archivo completo en memoria. Para archivos grandes, esa diferencia puede ser la que decide si el programa funciona o se queda sin memoria.

9.7. Escribir en un archivo

Para escribir tenés dos métodos, según tengas un texto suelto o una lista de textos:

Método	Qué hace	Ejemplo
`write(texto)`	Escribe un string	`f.write("Hola\n")`
`writelines(lista)`	Escribe todos los strings de una lista	`f.writelines(["a\n", "b\n"])`

```
archivo = open("notas.txt", "w")
archivo.write("Primera línea\n")
archivo.write("Segunda línea\n")
archivo.close()
```

Importante

Ni `write()` ni `writelines()` agregan el salto de línea por su cuenta. Si no ponés el `\n` vos, todo el texto queda pegado en un solo renglón. A diferencia de `print()`, que sí lo agrega solo, acá el salto es responsabilidad tuya.

Otra cosa importante: `write()` solo acepta **strings**. Si querés guardar un número tenés que convertirlo primero con `str()`, o Python te va a tirar un `TypeError`.

9.8. La forma recomendada: with

Si abrís un archivo, tenés que acordarte de cerrarlo, o pueden quedar datos sin guardar. La forma profesional y segura es usar `with`, que cierra el archivo automáticamente al terminar el bloque, **incluso si ocurre un error en el medio**.

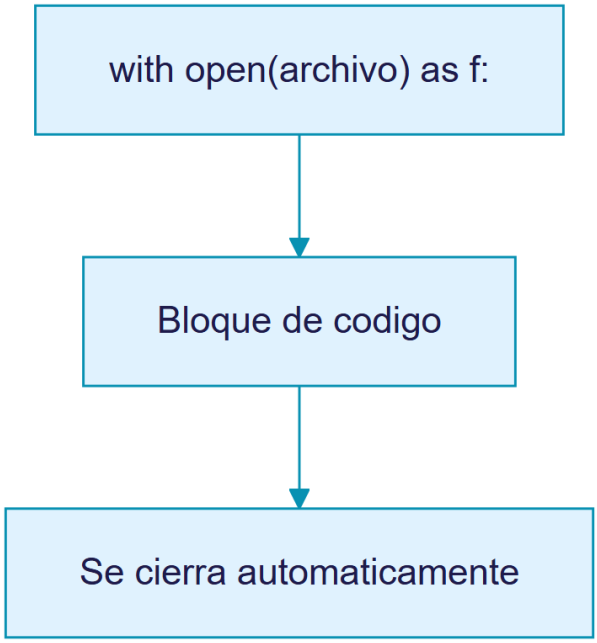


Diagrama 9.8: al salir del bloque with, el archivo se cierra solo.

```
with open("notas.txt", "w") as archivo:
    archivo.write("Primera línea\n")
    archivo.write("Segunda línea\n")

with open("notas.txt", "r") as archivo:
    for linea in archivo:
        print(linea.strip())
```

Qué hace el código:

- `with open("notas.txt", "w") as archivo:` abre el archivo en modo escritura y lo llama `archivo`.
- `archivo.write("...")` escribe una línea; el `\n` del final es un salto de línea (para que no quede todo pegado).
- Al salir del bloque `with`, el archivo **se cierra solo**: no hace falta acordarse de cerrarlo.
- El segundo `with` lo abre en modo lectura `"r"` y lo recorre línea por línea; `strip()` quita el salto de línea sobrante.

Analogía

Usar `with` es como esos grifos de baño público con cierre automático: no importa si te olvidás de cerrarlos, se cierran solos. Te evita el «goteo» de archivos abiertos que quedan ocupando recursos.

Buenas prácticas

Usá **siempre** `with` para trabajar con archivos. El `open()` + `close()` manual sirve para entender qué pasa por debajo, pero en el código que escribas de acá en adelante el `with` tiene que ser tu forma por defecto.

Encontrá el error

Este programa falla apenas intenta escribir. Fijate en qué modo se abrió el archivo antes de seguir.

```
archivo = open("resultado.txt", "r")
archivo.write("Nueva linea de resultados")
archivo.close()
```

Qué estaba mal

El archivo se abrió en modo "r" (solo lectura), así que no se puede escribir en él: Python lanza un error de operación no soportada. Para escribir hace falta abrirlo en modo "w" o "a", y lo recomendable es usar with para que se cierre solo.

Practicá la lectura

Explicá con tus palabras qué queda escrito en agenda.txt después de ejecutar este programa. ¿Cuántas líneas tiene el archivo al final?

```
with open("agenda.txt", "w") as f:
    f.write("Ana\n")
    f.write("Juan\n")

with open("agenda.txt", "a") as f:
    f.write("Lucia\n")

with open("agenda.txt", "w") as f:
    f.write("Pedro\n")
```

Para reflexionar y autoevaluarte

— ¿Qué ventaja concreta le ves a usar `with` en vez de abrir y cerrar el archivo a mano?

— ¿En qué situación usarías el modo `"a"` en lugar del `"w"`?

Antes de seguir, verificá que podés:

— Abrir, leer y escribir archivos usando `with`.

— Elegir el modo de apertura correcto (`r`, `w`, `a`) según lo que se necesita hacer.

— Recorrer un archivo línea por línea y explicar por qué conviene frente a `read()`.

— Acordarte de agregar el `\n` cuando escribís.

En la **Actividad 3** vas a usar todo esto para procesar archivos CSV: leer líneas, limpiarlas y separarlas en datos que tu programa pueda usar.